

A **governed agent** that fact-checks claims against the **live web** and ships a **cryptographically signed, verifiable receipt** — showing every source it admitted or rejected, and why.

LIVE [algorithm-reviews.vercel.app](#) REPO [github.com/OrionArchitekton/algorithm-reviews](#)

Overall

Nimble · live web

name.com · Domain Roulette

01 The problem

AI answers and online reviews are **unauditable**: you can't tell a grounded verdict from a hallucinated one, and sources are stale, fabricated, or cherry-picked. Buyers, journalists, compliance, and due-diligence teams who must **defend a conclusion** need a citation trail and a timestamp — not a vibe.

02 Architecture — two planes, one app

One Next.js app, two cleanly separated planes. The **execution plane** searches and extracts live-web evidence; the **governance plane** decides what evidence is *admissible* (fail-closed), adjudicates the verdict, and signs the receipt.



☐ Execution plane — act ☐ Governance plane — decide ☒ Admissibility gate

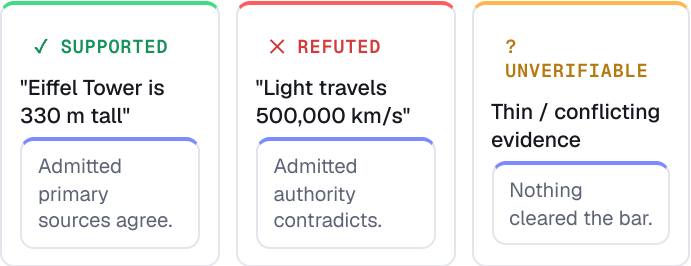
The model proposes **signals** (authority, relevance). **POLICY — not the model — decides admission**, fail-closed and deterministic. Every **admit** / **reject** a judge sees is a unit-tested code path, not a prompt.

03 The governance plane — admissibility policy

A deterministic, unit-tested policy (`lib/agent/govern.ts`) scores each source against the claim. Gates run in order — the **first failing gate wins**, so the reason shown is the primary defect.

- reject** **Content** — no extractable snippet/extract.
- reject** **Relevance** < 0.45 — off-topic for this claim.
- reject** **Authority** < 0.35 — low-trust / unsourced.
- reject** **Freshness** > 540d stale — *unless* authority ≥ 0.85 (primary records don't rot).
- admit** **Cleared** — reason states authority, relevance & freshness.

Fail-closed. A claim that ends with *zero* admitted sources resolves to **unverifiable** — never **false**. Absence of proof ≠ proof of absence.



04 Verifiable receipts — the headline

Every receipt is signed with **ECDSA P-256** (Web Crypto `crypto.subtle.sign`) over a **deterministic canonical payload**. The canonical projection *excludes* volatile fields (`createdAt`, `fetchAt`, `ordering`), sorts claims & citations, and rounds confidences — so re-running against an unchanged web yields an **identical signature**; only a moved verdict changes it. The public key is published at `/api/pubkey`; `/verify` recomputes the payload and checks the signature.

Verify it yourself. Tamper with one field on `/verify` and the signature fails — tamper detection is exercised by the test suite. "Signed review receipt" is *literally* true.

05 Live web & resilience

- Swappable **WebProvider** interface (`search` + `extract`). Primary: **Nimble** Search + Extract — REST `POST` `sdk.nimbleway.com/v1/*`, Bearer auth.
- Keyless Wikipedia fallback** — datacenter-IP search (DuckDuckGo HTML, `s.jina.ai`) is blocked/keyed from Vercel, so full-article Wikipedia extracts keep live reviews real today.
- FallbackProvider auto-degrades** on error/empty; the receipt records which provider actually answered.
- Deterministic mock** + `/demo` **fixtures** (`link-rot`, `nimble-best`, `claude-context`) — zero-network, can't rate-limit or 500 on stage.

06 Engineering rigor

The **model layer resolves** Anthropic key → Vercel AI Gateway (OIDC) → deterministic mock, so it **builds, tests, and demos with zero keys** and goes live the instant a key or the gateway is available — no code change. **19 Vitest unit tests** cover the admissibility policy (incl. the stale-but-high-authority override & fail-closed-on-no-content), receipt signing / tamper detection / determinism, and the UI reducer. **typecheck + lint + build green.**

Next.js 16 React 19 Tailwind v4 TypeScript

Vercel AI SDK 6 Claude Opus · adjudication

Claude Haiku · classify Web Crypto · ECDSA

Vitest · 19 tests

07 Three challenges, one build

Overall

A real, recurring, monetizable wedge — trust & safety, due-diligence, procurement.

Nimble

The live-web dependency is load-bearing & visible: admit/reject of live sources via Search + Extract.

name.com · Roulette

A literal structural read of `algorithm.reviews` — an algorithm that reviews, reviewably.

Tower cut, deliberately. Non-cash, and a second-sponsor onboarding + lakehouse on the critical path was the highest-variance time sink for a 3.5-day solo build — so it was scoped out to protect the live demo.

08 Try it in 60 seconds

- Open **algorithm-reviews.vercel.app** and paste `The Eiffel Tower is 330 metres tall` → **SUPPORTED** with a live citation.
- Paste `the speed of light is 500,000 km/s` → **REFUTED** with a contradicting citation.
- Go to `/verify`, hit **"Tamper with it"**, re-verify → the signature **fails**.
- Offline-safe on stage: open `/demo/link-rot` (zero network). Locally: `npm install && npm run dev` works keyless in mock mode.

REVIEW RECEIPT

✓ Supported · 97%

```
• signed   alg ECDSA_P-256_SHA-256   key 3rR_9Arv...
sha256 6f5a18e4bcd9...   web nimble+web-fallback
model anthropic-direct
```

canonical hash matches: yes · key id matches server: yes · signature verifies: yes

Hard caps in code (`lib/agent/caps.ts`), not prompts — the single most important defense for a live demo. They keep any run inside provider rate limits *and* the Vercel function budget.

3

max atomic claims

2

queries / claim

8

total extracts

5

candidates / claim

2

web concurrency

15s

request timeout

SHAPED BY ADVERSARIAL REVIEW · 5/10 → 7.5/10

"Signed must mean signed."

A bare SHA-256 hash was upgraded to a real ECDSA P-256 signature with a public key + verify page.

"You can't search from a datacenter IP."

DuckDuckGo HTML returned 0 results from Vercel → added the keyless Wikipedia provider so live reviews still produce admitted sources.